

第八章 Clean Code（面试突击★★★★）

编写整洁代码不是炫技，而是一种工程素养。本章围绕命名、函数、SOLID、内聚耦合、代码异味、重构以及C++特化实践，提供系统化的面试准备。

目录

1. [命名规范](#)
2. [函数设计](#)
3. [SOLID原则](#)
4. [高内聚低耦合](#)
5. [代码异味](#)
6. [重构](#)
7. [C++ Specific Clean Code](#)
8. [面试核心](#)

1. 命名规范

命名的黄金法则：**名字应该告诉你它为什么存在、做什么事、该怎么用。**

1.1 变量命名

原则：有意义的、可读的、可搜索的

BAD vs GOOD 示例

示例1：避免无意义缩写

```
// BAD
int d;           // 经过多少天？什么是 d？
int elpsd;       // 非标准缩写

// GOOD
int daysSinceCreation;
int elapsedTimeInSeconds;
```

示例2：区分相似概念

```
// BAD - 极度相似，容易混淆
int fileCount;
int fileCnt;      // 与上面几乎一样

// GOOD - 语义清晰
int totalFileCount;
int processedFileCount;
```

示例3: 避免单字母 (除循环变量外)

```
// BAD
auto p = getPerson();    // p 是什么?
auto d = calculate();    // d 是 date? distance? data?

// GOOD
auto person = getPerson();
auto distance = calculate();
```

示例4: 布尔变量命名

```
// BAD
bool flag = true;        // flag 是什么标志?
bool status = false;     // 什么状态?

// GOOD
bool isConnected = true;
bool hasError = false;
bool canRetry = true;
```

示例5: 使用领域术语

```
// BAD - 脱离业务
int a;
double b;

// GOOD - 使用业务术语
int accountBalance;
double interestRate;
```

示例6: 魔法数字 vs 命名常量

```
// BAD
if (employee.yearsOfService > 5) { ... }
for (int i = 0; i < 86400; ++i) { ... }
double tax = income * 0.17;

// GOOD
constexpr int YEARS_FOR_SENIORITY = 5;
if (employee.yearsOfService > YEARS_FOR_SENIORITY) { ... }

constexpr int SECONDS_PER_DAY = 86400;
for (int i = 0; i < SECONDS_PER_DAY; ++i) { ... }

constexpr double TAX_RATE = 0.17;
double tax = income * TAX_RATE;
```

示例7: 容器命名用复数

```
// BAD
std::vector<Order> order;

// GOOD
std::vector<Order> orders;
std::unordered_map<int, Customer> customerMap; // 类型提示
```

1.2 函数命名

原则：动词 + 名词

```
// BAD
void data();           // 不表明操作
void process();        // 太泛
int x();

// GOOD
void saveData();
void processPayment();
int getAccountBalance();
void updateCustomerAddress();
bool validateEmailFormat();
```

函数命名常用动词：

get/set, create/build, save/persist,
load/fetch, delete/remove, calculate/compute,
validate/check, transform/convert, send/dispatch,
initialize/setup, finalize/teardown

1.3 类命名

原则：名词或名词短语

```
// BAD
class DoSomething;      // 这是函数名风格
class Data;            // 太泛
class Manager;         // 太泛，什么的管理者？

// GOOD
class Customer;
class AccountRepository;
class EmailSender;
class JsonSerializer;
```

1.4 应避免的命名风格

应避免	原因	替代方案
匈牙利命名法 <code>iCount</code> , <code>szName</code>	IDE 已经能显示类型	<code>count</code> , <code>name</code>
前缀下划线 <code>_value</code>	标准库保留	<code>value_</code> (后缀)
双下划线 <code>__id</code>	编译器保留	<code>id_</code>
前缀 <code>m_</code>	历史产物, 现代C++不推荐	<code>_</code> 后缀或不用前缀
混用大小写 <code>myVar</code> / <code>my_var</code>	不一致	统一风格 (<code>snake_case</code>)
拼音命名	无法国际化	英文术语

1.5 匈牙利命名法批判

```
// 匈牙利命名法 (BAD)
int iCount;           // i 表示 int
char* szName;         // sz 表示 null-terminated string
bool bIsValid;        // b 表示 bool
float fPrice;          // f 表示 float

// 现代 C++ (GOOD)
int count;             // IDE 悬停即可看到类型
std::string name;
bool isValid;
float price;
```

为什么淘汰？

- 1. IDE 普及：鼠标悬停即见类型，无需编码在名字里
- 2. 类型会变：`int iCount` 改成 `long` 时名字要改吗？
- 3. 噪声：前缀 = 冗余信息
- 4. 抽象性差：`std::string` / `std::vector<int>` 用什么前缀？

2. 函数设计

"函数的第一规则是短小，第二规则是更短小。" — Robert C. Martin

2.1 短小，只做一件事

```
// BAD - 一个函数做了三件事：验证、格式化、保存
void processUserData(const std::string& name, int age) {
    // 验证
    if (name.empty() || age < 0) {
        throw std::invalid_argument("Invalid user data");
    }
}
```

```

// 格式化
std::string formatted = name + " (" + std::to_string(age) + ")";
// 保存
database.save(formatted);
}

// GOOD - 三个函数，每个只做一件事
void validateUserData(const std::string& name, int age) {
    if (name.empty()) throw std::invalid_argument("name is empty");
    if (age < 0) throw std::invalid_argument("age is negative");
}

std::string formatUserData(const std::string& name, int age) {
    return name + " (" + std::to_string(age) + ")";
}

void saveFormattedUser(const std::string& formatted) {
    database.save(formatted);
}

void processUserData(const std::string& name, int age) {
    validateUserData(name, age);
    auto formatted = formatUserData(name, age);
    saveFormattedUser(formatted);
}

```

2.2 函数参数：最多3个

```

// BAD - 5个参数，调用时容易顺序错乱
void createUser(const std::string& name, int age,
               const std::string& email, const std::string& phone,
               const std::string& address);

// GOOD - 参数过多 → 封装成 struct/class
struct UserInfo {
    std::string name;
    int age;
    std::string email;
    std::string phone;
    std::string address;
};

void createUser(const UserInfo& info);

```

2.3 避免布尔参数

```
// BAD - 布尔参数让调用点语意不清
render(true);    // true 是什么意思?

// GOOD - 拆成两个函数, 或用枚举
void renderWithShadows();
void renderWithoutShadows();

// 或者用枚举
enum class RenderMode { WithShadows, WithoutShadows };
void render(RenderMode mode);
```

2.4 无输出参数, 返回值

```
// BAD - 输出参数 (out-parameter)
void getUserData(std::string& name, int& age) {
    name = database.getName();
    age = database.getAge();
}

// GOOD - 返回结构体
struct UserData {
    std::string name;
    int age;
};
UserData getUserData() {
    return {database.getName(), database.getAge()};
}
```

2.5 命令-查询分离 (CQS)

```
// BAD - 一个函数既修改状态又返回结果
bool setAttribute(const std::string& key, const std::string& value) {
    if (attributes_[key] == value) return false; // 查询
    attributes_[key] = value;                    // 命令
    return true;                                 // 查询
}

// GOOD - 分离查询和命令
bool hasAttribute(const std::string& key) const { // 查询
    return attributes_.count(key) > 0;
}
bool attributeEquals(const std::string& key,
                    const std::string& value) const { // 查询
    auto it = attributes_.find(key);
    return it != attributes_.end() && it->second == value;
}
void setAttribute(const std::string& key,
                 const std::string& value) { // 命令
```

```
    attributes_[key] = value;
}
```

2.6 提前返回 vs 单一返回

```
// BAD - 深层嵌套（箭头代码）
bool validateOrder(const Order& order) {
    if (order.isValid()) {
        if (order.hasItems()) {
            if (order.totalAmount() > 0) {
                if (customerExists(order.customerId())) {
                    return true;
                } else {
                    logError("No customer");
                    return false;
                }
            } else {
                logError("Amount <= 0");
                return false;
            }
        } else {
            logError("No items");
            return false;
        }
    } else {
        logError("Invalid order");
        return false;
    }
}

// GOOD - Guard Clause (卫语句): 提前返回, 消除 else
bool validateOrder(const Order& order) {
    if (!order.isValid()) {
        logError("Invalid order");
        return false;
    }
    if (!order.hasItems()) {
        logError("No items");
        return false;
    }
    if (order.totalAmount() <= 0) {
        logError("Amount <= 0");
        return false;
    }
    if (!customerExists(order.customerId())) {
        logError("No customer");
        return false;
    }
    return true;
}
```

面试话术: "我使用卫语句 (Guard Clause) 提前处理异常/边界情况, 让主逻辑保持在最低缩进层级, 提高可读性。"

2.7 更多 BAD vs GOOD 示例

```
// BAD - 副作用隐藏在 getter 中
int getCount() {
    refreshCache(); // 隐藏的副作用!
    return count_;
}

// GOOD - 副作用可见
void refreshCache() { /* ... */ }
int getCount() const { return count_; }

// BAD - 函数名说谎
void deleteUser(int id) {
    user.setDeleted(true); // 软删除, 没有真正删除
}

// GOOD - 函数名诚实
void softDeleteUser(int id) {
    user.setDeleted(true);
}

// BAD - 过长的函数 (>30 行通常有提取空间)
void processRequest(...) {
    // 200 行代码在一屏看不到头尾
}

// GOOD - 提取小函数, 每个不超过 20 行
void processRequest(...) {
    auto parsed = parseRequest(raw);
    auto validated = validateRequest(parsed);
    auto result = handleRequest(validated);
    sendResponse(result);
}
```

3. SOLID原则

SOLID 是面向对象设计的五大基本原则, 由 Robert C. Martin 提出。面试必问。

3.1 S — 单一职责原则 (Single Responsibility Principle)

定义: 一个类应该只有一个引起它变化的原因。一个模块应该只对一个角色负责。

为什么重要?

- 减少耦合: 修改某个职责不影响其他职责
- 易于理解和测试: 职责单一 → 代码量小 → 理解快
- 降低合并冲突风险: 不同团队修改不同职责, 互不干扰

BAD 示例:


```
// 违反 SRP: 一个类承担了数据存储、格式化、输出的三个职责
class Report {
public:
    void loadData();           // 数据存取
    void calculateStats();     // 计算逻辑
    void formatHTML();         // HTML 格式化
    void formatPDF();          // PDF 格式化
    void print();              // 打印/输出
    void email();              // 邮件发送
    // 每次因为任何原因修改都影响同一个类
};
```

GOOD 示例:

```
class ReportData {
public:
    void loadData();
    void calculateStats();
};

class ReportFormatter {
public:
    virtual std::string format(const ReportData& data) = 0;
};

class HtmlFormatter : public ReportFormatter {
public:
    std::string format(const ReportData& data) override;
};

class PdfFormatter : public ReportFormatter {
public:
    std::string format(const ReportData& data) override;
};

class ReportPrinter {
public:
    void print(const std::string& content);
};

class ReportMailer {
public:
    void send(const ReportData& data, const std::string& formatted);
};
```

面试题 Q: 如何判断一个类是否违反 SRP?

反过来读职责描述。"这个类负责 [职责A] 和 [职责B]" —— 有"和"字就是违反信号。另一个快速检测法是问: "这个类的修改原因有几个?" 如果多于一个, 就违反 SRP。

3.2 O — 开闭原则 (Open-Closed Principle)

定义：软件实体（类、模块、函数）应该对扩展开放，对修改关闭。即：新增功能通过扩展实现，而非修改已有代码。

为什么重要？

- 避免回归 bug：不改已有代码，已有功能不会坏
- 降低风险：新增功能 = 新增文件
- 容易 code review：只 review 新增代码

BAD 示例：

```
// 违反 OCP: 每次新增图形都要修改 switch
enum class ShapeType { Circle, Rectangle, Triangle };

class AreaCalculator {
public:
    double calculateArea(ShapeType type, double param1, double param2 = 0) {
        switch (type) {
            case ShapeType::Circle:
                return 3.14159 * param1 * param1;
            case ShapeType::Rectangle:
                return param1 * param2;
            case ShapeType::Triangle:
                return param1 * param2 / 2;
            // 新增图形 → 修改此处 → 违反 OCP
            default:
                return 0;
        }
    }
};
```

GOOD 示例：

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
public:
    explicit Circle(double r) : radius_(r) {}
    double area() const override { return 3.14159 * radius_ * radius_; }
private:
    double radius_;
};

class Rectangle : public Shape {
public:
    Rectangle(double w, double h) : width_(w), height_(h) {}
```

```

    double area() const override { return width_ * height_; }
private:
    double width_, height_;
};

// 新增图形：只需新增一个类，不改已有代码
class Triangle : public Shape {
public:
    Triangle(double b, double h) : base_(b), height_(h) {}
    double area() const override { return base_ * height_ / 2; }
private:
    double base_, height_;
};

double totalArea(const std::vector<std::unique_ptr<Shape>>& shapes) {
    double sum = 0;
    for (const auto& s : shapes) sum += s->area();
    return sum;
}

```

面试题 Q：开闭原则就是多态吗？

不完全是。多态是实现开闭原则的一种方式，但不是唯一方式。其他方式包括：

- 策略模式 (Strategy Pattern)
- 插件架构
- 配置文件/规则引擎
- 模板特化

关键是思想：通过抽象和多态将变化点外移。

3.3 L — 里氏替换原则 (Liskov Substitution Principle)

定义：子类型必须能够替换其基类型而不破坏程序的正确性。（Barbara Liskov, 1987）

通俗说法：**凡是父类出现的地方，子类都可以无缝替代。**

为什么重要？

- 保证多态的正确性
- 违反 LSP 会导致调用者代码需要 `dynamic_cast` 或特殊判断
- 是继承的前提条件

BAD 示例 (违反 LSP)：

```

class Rectangle {
public:
    virtual void setwidth(int w) { width_ = w; }
    virtual void setHeight(int h) { height_ = h; }
    int getwidth() const { return width_; }
    int getHeight() const { return height_; }
    int area() const { return width_ * height_; }
protected:

```

```

    int width_ = 0;
    int height_ = 0;
};

class Square : public Rectangle {
public:
    // 违反 LSP: 修改了 setWidth 的语义
    // 父类 setWidth 只改宽度, 子类却连高度也改了
    void setWidth(int w) override {
        width_ = w;
        height_ = w;    // 正方形变宽度→高度也得变
    }
    void setHeight(int h) override {
        width_ = h;    // 同上
        height_ = h;
    }
};

// 问题暴露
void testRectangle(Rectangle& r) {
    r.setWidth(5);
    r.setHeight(4);
    assert(r.area() == 20); // 对 Rectangle 成立, 对 Square 是 16!
}

```

GOOD 示例:

```

// 方案1: 不要用继承, 正方形和长方形分别实现 Shape
class Shape {
public:
    virtual int area() const = 0;
    virtual ~Shape() = default;
};

class Rectangle : public Shape {
public:
    Rectangle(int w, int h) : width_(w), height_(h) {}
    int area() const override { return width_ * height_; }
    void setWidth(int w) { width_ = w; }
    void setHeight(int h) { height_ = h; }
private:
    int width_, height_;
};

class Square : public Shape {
public:
    explicit Square(int side) : side_(side) {}
    int area() const override { return side_ * side_; }
    void setSide(int s) { side_ = s; }
private:
    int side_;
};

```

面试题 Q：如何判断继承关系是否违反 LSP？

三条标准：

1. **前置条件不能加强**（子类不能比父类要求更多 — 参数限制不能更严）
2. **后置条件不能削弱**（子类不能比父类做得更少 — 返回值保证不能更弱）
3. **不变量要保持**（父类维护的不变量，子类也必须维护）

3.4 I — 接口隔离原则 (Interface Segregation Principle)

定义：客户端不应该被迫依赖它不使用的方法。大接口应该拆分成更小、更具体的接口。

为什么重要？

- 避免实现类有冗余的空方法（实现不需要的功能）
- 减少耦合：调用方只依赖需要的接口
- 让接口更内聚

BAD 示例：

```
// 违反 ISP: 一个肥胖的接口
class IWorker {
public:
    virtual void work() = 0;
    virtual void eat() = 0;
    virtual void sleep() = 0;
    virtual ~IWorker() = default;
};

class Human : public IWorker {
public:
    void work() override { /* ... */ }
    void eat() override { /* ... */ }
    void sleep() override { /* ... */ }
};

class Robot : public IWorker {
public:
    void work() override { /* ... */ }
    void eat() override { /* 机器人不吃东西！空实现 */ }
    void sleep() override { /* 机器人不睡觉！空实现 */ }
};
```

GOOD 示例：

```
class IWorkable {
public:
    virtual void work() = 0;
    virtual ~IWorkable() = default;
};

class IEatable {
```

```

public:
    virtual void eat() = 0;
    virtual ~IEatable() = default;
};

class ISleepable {
public:
    virtual void sleep() = 0;
    virtual ~ISleepable() = default;
};

class Human : public IWorkable, public IEatable, public ISleepable {
public:
    void work() override { /* ... */ }
    void eat() override { /* ... */ }
    void sleep() override { /* ... */ }
};

class Robot : public IWorkable {
public:
    void work() override { /* ... */ }
    // 不实现 eat/sleep
};

```

面试题 Q：接口隔离和单一职责有什么区别？

- **SRP** 关注类的职责单一："这个类有几个修改原因？"
- **ISP** 关注接口的粒度："这个接口的方法是否都被需要？"

两者互补：SRP 让类专注，ISP 让接口精简。

3.5 D — 依赖倒置原则 (Dependency Inversion Principle)

定义：

1. 高层模块不应该依赖低层模块，两者都应该依赖抽象
2. 抽象不应该依赖具体实现，具体实现应该依赖抽象

为什么重要？

- 解耦：高层业务逻辑不依赖底层技术细节
- 可测试：依赖接口可以轻松 mock
- 可替换：切换底层实现不需要改高层代码

BAD 示例：

```

// 违反 DIP: 高层 (OrderService) 直接依赖低层 (MySQLDatabase)
class MySQLDatabase {
public:
    void save(const std::string& data) {
        // 直接操作 MySQL
    }
    std::string load(int id) {

```

```

        return "data from MySQL";
    }
};

class OrderService {
public:
    // 紧耦合 MySQLDatabase
    void createOrder(const std::string& order) {
        db_.save(order);
    }
private:
    MySQLDatabase db_; // 高层直接依赖低层具体类
};

```

GOOD 示例:

```

class IDatabase {
public:
    virtual void save(const std::string& data) = 0;
    virtual std::string load(int id) = 0;
    virtual ~IDatabase() = default;
};

class MySQLDatabase : public IDatabase {
public:
    void save(const std::string& data) override { /* ... */ }
    std::string load(int id) override { return "data"; }
};

class PostgreSQLDatabase : public IDatabase {
public:
    void save(const std::string& data) override { /* ... */ }
    std::string load(int id) override { return "data"; }
};

class OrderService {
public:
    // 依赖抽象接口 - 构造函数注入
    explicit OrderService(std::unique_ptr<IDatabase> db)
        : db_(std::move(db)) {}
    void createOrder(const std::string& order) { db_>save(order); }
private:
    std::unique_ptr<IDatabase> db_;
};

// 使用
auto service = OrderService(std::make_unique<MySQLDatabase>());
// 切换数据库: 只需换注入的对象, OrderService 代码不变
auto service2 = OrderService(std::make_unique<PostgreSQLDatabase>());

```

面试题 Q: 依赖注入和依赖倒置的关系?

依赖注入是实现依赖倒置**最常见的手段**, 但两者不等同:

- **DIP 是原则**（依赖抽象而非具体）
- **DI 是模式**（从外部注入依赖，而非内部创建）

其他实现 DIP 的方式：服务定位器（Service Locator）、工厂模式、插件架构。

4. 高内聚低耦合

软件工程的核心：让紧密相关的东西放在一起（高内聚），让无关的东西互不依赖（低耦合）。

4.1 内聚类型（从低到高）

级别	类型	描述	示例
1 (最低)	偶然内聚	元素随机放在一起	一个 Utility 类里放了字符串函数、文件函数、数学函数
2	逻辑内聚	逻辑上属于一类但功能不同	一个 I/O 类里同时处理文件、网络、串口
3	时间内聚	同一时间执行	Init() 里初始化所有系统（不管是否相关）
4	过程内聚	按执行顺序	一个函数按顺序调用不相关的步骤
5	通信内聚	操作同一数据	一个类所有方法都操作同一个文件
6	顺序内聚	输出作为下一步输入	读文件 → 解析 → 存储
7 (最高)	功能内聚	所有元素完成一个单一功能	Math::sin(), Math::cos()

面试重点：能说出功能内聚是最理想的，并举例说明。

```
// 低内聚 - 偶然内聚：什么都放的 Utility 类
class MiscUtils {
public:
    static std::string toUpper(const std::string& s);    // 字符串
    static void deleteFile(const std::string& path);    // 文件
    static double sqrt(double x);                      // 数学
    static void sendEmail(const std::string& to);        // 邮件
};

// 高内聚 - 功能内聚：每个类职责单一
class StringUtils {
public:
    static std::string toUpper(const std::string& s);
};
```



```
class FileSystem {
public:
    void deleteFile(const std::string& path);
};

class MathUtils {
public:
    static double sqrt(double x);
};
```

4.2 耦合类型（从紧到松）

级别	类型	描述	示例
1 (最紧)	内容耦合	一个模块修改另一个模块内部数据	直接访问另一个类的 private 成员
2	公共耦合	共享全局变量	多个模块读写同一个全局变量
3	控制耦合	通过控制标志影响另一个模块	传入 flag 参数控制行为
4	标记耦合	通过复杂数据结构的部分字段传递	传入整个 struct 但只用其中2个字段
5	数据耦合	仅通过简单数据（参数/返回值）	传入 int, string, 返回 bool
6 (最松)	无耦合	模块间无任何联系	完全独立的模块

```
// 紧耦合 - 内容耦合 / 公共耦合
int globalCounter = 0; // 公共耦合：谁都能改

class BadModule {
public:
    void dowork() {
        globalCounter++; // 修改全局变量
        otherModule_.privateVar = 5; // 内容耦合：访问别的对象内部
    }
private:
    OtherModule otherModule_;
};

// 松耦合 - 数据耦合
class GoodModule {
public:
    // 只通过参数和返回值交互
    int dowork(int input) {
        return input + 1;
    }
};
```

4.3 迪米特法则 (Law of Demeter)

定义：一个对象应该对其他对象有尽可能少的了解。只和最亲近的朋友说话。

简化规则：方法 M 只能调用以下对象的方法：

1. 自身
2. 方法参数
3. 自身创建的对象
4. 自身直接持有的成员对象

```
// BAD - 违反 Demeter: 链式调用 (火车残骸)
void printManagerName(const Department& dept) {
    // 一路 . 下去, 知道太多内部结构
    std::cout << dept.getManager().getAddress().getCity() << std::endl;
}

// GOOD - 只和直接朋友说话
void printManagerCity(const Department& dept) {
    std::cout << dept.getManagerCity() << std::endl; // Dept 封装细节
}

// 部门内部封装
class Department {
public:
    std::string getManagerCity() const {
        return manager_.getCity(); // 实现细节, 外部不需要知道
    }
private:
    Manager manager_;
};
```

5. 代码异味 (Code Smells)

代码异味是代码中可能存在问题的表面迹象。以下列出 20+ 种常见异味，每种配 BAD vs GOOD。

5.1 Long Method (过长函数)

```
// BAD - 200 行函数
void processOrder() {
    // 200 lines of validation, calculation, formatting, saving...
}

// GOOD - 提取为多个小函数
void processOrder() {
    validateOrder();
    auto price = calculatePrice();
    auto receipt = formatReceipt(price);
    saveOrder(receipt);
    notifyCustomer();
}
```

5.2 Large Class (上帝类)

```
// BAD - 一个类做所有事
class OrderManager { // 3000 lines
    // CRUD, 校验, 价格计算, 打印, 邮件, 报表...
};

// GOOD - 按职责拆分
class OrderRepository { /* CRUD */ };
class OrderValidator { /* 校验 */ };
class PriceCalculator { /* 价格 */ };
class ReceiptPrinter { /* 打印 */ };
```

5.3 Long Parameter List (长参数列表)

```
// BAD
void createUser(string name, string email, string phone,
               string addr, string city, string zip,
               int age, bool vip, string note);

// GOOD
struct CreateUserRequest {
    string name, email, phone, address, city, zip;
    int age;
    bool vip;
    string note;
};

void createUser(const CreateUserRequest& req);
```

5.4 Primitive Obsession (基本类型偏执)

```
// BAD - 用 string 表达一切
void makeReservation(string date,      // "2024-01-15"
                    string time,      // "14:30"
                    string amount);   // "100.00"

// GOOD - 封装为领域类型
class Date { /* 内置验证 */ };
class Time { /* 内置验证 */ };
class Money { /* 内置格式化和运算 */ };
void makeReservation(Date date, Time time, Money amount);
```

5.5 Data Clumps (数据泥团)

```
// BAD - start/end 总是成对出现
void drawRect(int x, int y, int w, int h);
int calculateDistance(int startX, int startY, int endX, int endY);

// GOOD - 提取为类
struct Point { int x, y; };
struct Size { int width, height; };
struct Rect { Point origin; Size size; };
void drawRect(const Rect& rect);
int calculateDistance(const Point& start, const Point& end);
```

5.6 Switch Statements (用多态替代 switch)

```
// BAD - 基于类型的 switch
double getPay(EmployeeType type) {
    switch (type) {
        case ENGINEER: return salary_ + bonus_;
        case MANAGER:  return salary_ + bonus_ + stock_;
        case SALES:    return salary_ + commission_;
    }
}

// GOOD - 用多态消除 switch
class Employee {
public:
    virtual double getPay() const = 0;
};
class Engineer : public Employee {
    double getPay() const override { return salary_ + bonus_; }
};
class Manager : public Employee {
    double getPay() const override { return salary_ + bonus_ + stock_; }
};
```

5.7 Feature Envy (依恋情结)

```
// BAD - Phone 的方法大量访问 Address 的数据
class Phone {
public:
    string getFullAddress() {
        // 过度依赖 Address 的数据
        return address_.getStreet() + ", " +
            address_.getCity() + " " + address_.getZip();
    }
private:
    Address address_;
};

// GOOD - 把行为移到它依赖的数据所在类
class Address {
public:
    string getFullAddress() const {
        return street_ + ", " + city_ + " " + zip_;
    }
};
```

5.8 Inappropriate Intimacy (过度亲密)

```
// BAD - 两个类过分了解彼此内部
class A {
    friend class B; // B 可以访问 A 的 private
};

// GOOD - 只暴露必要的公开接口
class A {
public:
    int getValue() const { return value_; }
    void setValue(int v) { value_ = v; }
private:
    int value_;
};
```

5.9 Comments as Deodorant (注释作除臭剂)

```
// BAD - 代码不清，用注释掩盖
// 如果用户是 VIP 且订单总额 > 1000 且不是周末
// 并且库存足够，就应用折扣
if (user.isvip() && order.total > 1000 && !isweekend()
    && inventory.hasStock(order.itemId)) {
    order.applyDiscount(0.9);
}

// GOOD - 用命名清晰的函数替代注释
if (userQualifiesForDiscount(user, order)) {
```

```

    order.applyDiscount(0.9);
}

bool userQualifiesForDiscount(const User& user, const Order& order) {
    return user.isvip()
        && order.total > DISCOUNT_THRESHOLD
        && !isweekend()
        && inventory.hasStock(order.itemId);
}

```

5.10 Duplicate Code (重复代码)

```

// BAD - 同样的逻辑写了两遍
void processAdmin(const User& u) {
    if (u.isActive() && u.hasPermission("admin")) {
        grantAccess(u);
        logAccess(u);
        sendNotification(u);
    }
}

void processEditor(const User& u) {
    if (u.isActive() && u.hasPermission("editor")) {
        grantAccess(u);
        logAccess(u);
        sendNotification(u);
    }
}

// GOOD - 提取公共逻辑
void processUser(const User& u, const std::string& requiredPermission) {
    if (u.isActive() && u.hasPermission(requiredPermission)) {
        grantAccess(u);
        logAccess(u);
        sendNotification(u);
    }
}

void processAdmin(const User& u) { processUser(u, "admin"); }
void processEditor(const User& u) { processUser(u, "editor"); }

```

5.11 Dead Code (死代码)

```

// BAD - 从未被调用的函数、注释掉的代码、无法到达的分支
void legacyFunction() { /* 从 2019 年起就没人调用了 */ }
// if (false) { oldLogic(); } // 注释掉的代码

// GOOD - 删除! 版本控制系统里有历史
// 删除。不要注释掉。相信 Git。

```

5.12 Speculative Generality (夸夸其谈未来性)

```
// BAD - "将来可能需要" 的过度设计
class AbstractGenericDataProcessingPipelineFactory { // 只有一个实现
    virtual void preProcess() = 0;
    virtual void process() = 0;
    virtual void postProcess() = 0;
    virtual void prePreProcess() = 0; // "将来可能用到"
};

// GOOD - 先写简单实现，有需求再抽象
class OrderProcessor {
    void process(const Order& order) {
        // 现在只需要这个
    }
};

// YAGNI: You Aren't Gonna Need It
```

5.13 其他常见代码异味

异味	描述	解决方法
发散式变化	一个类因不同原因被修改	拆分
霰弹式修改	一个变化需要改很多类	合并
平行继承体系	每加一个子类 A 就要加子类 B	让一个体系的实例引用另一个
冗余类	类除了委托不做任何事	内联或删除
临时字段	字段只在某些时候有意义	提取到新类
消息链	a.b().c().d()	隐藏委托
中间人	类过度委托	移除中间人
异曲同工类	两个类做类似但接口不同	统一接口
不完美的库类	库不够好用	包装或引入新方法
纯数据类	只有 getter/setter	合并数据和行为

6. 重构 (Refactoring)

重构：在不改变外部行为的前提下，改善代码内部结构。

6.1 何时重构

时机	说明
添加功能前	重构让新功能更容易添加
Code Review 时	评审发现异味就改
修复 Bug 后	让代码更清晰以防同类 bug
理解代码后	用重构记录你的理解
定期	技术债偿还周期

不重构的情况：

- 工作正常的代码
- Deadline 前（先记到 todo）
- 没有测试保护的代码（先写测试再重构）
- 外部接口的破坏性修改

6.2 常用重构手法

Extract Method (提取函数)

```
// BEFORE - 长函数
void printOwing() {
    // Print banner
    std::cout << "*****" << std::endl;
    std::cout << "*** Customer Owes ***" << std::endl;
    std::cout << "*****" << std::endl;

    double outstanding = 0;
    for (const auto& order : orders_) {
        outstanding += order.amount;
    }

    // Print details
    std::cout << "name: " << name_ << std::endl;
    std::cout << "amount: " << outstanding << std::endl;
}

// AFTER - 提取子函数
void printOwing() {
    printBanner();
    double outstanding = calculateOutstanding();
    printDetails(outstanding);
}

void printBanner() {
    std::cout << "*****" << std::endl;
    std::cout << "*** Customer Owes ***" << std::endl;
}
```



```

        std::cout << "*****" << std::endl;
    }

    double calculateOutstanding() {
        double result = 0;
        for (const auto& order : orders_) result += order.amount;
        return result;
    }

    void printDetails(double outstanding) {
        std::cout << "name: " << name_ << std::endl;
        std::cout << "amount: " << outstanding << std::endl;
    }
}

```

Replace Conditional with Polymorphism (以多态取代条件)

```

// BEFORE - 基于类型的条件判断
double getSpeed(VehicleType type) {
    switch (type) {
        case CAR:      return 120;
        case TRUCK:    return 80;
        case BICYCLE:  return 25;
        default:       return 0;
    }
}

// AFTER - 多态
class Vehicle {
public:
    virtual double getSpeed() const = 0;
    virtual ~Vehicle() = default;
};

class Car : public Vehicle { double getSpeed() const override { return 120; } };
class Truck : public Vehicle { double getSpeed() const override { return 80; } };
class Bicycle : public Vehicle { double getSpeed() const override { return 25; } };

```

Introduce Parameter Object (引入参数对象)

```

// BEFORE - 参数列表过长
void createReservation(string name, string date, string time,
                      int partySize, string phone);

// AFTER - 参数对象
struct ReservationRequest {
    string customerName;
    string date;
    string time;
    int partySize;
    string phoneNumber;
};

void createReservation(const ReservationRequest& req);

```

Replace Magic Number with Symbolic Constant

```
// BEFORE - 魔法数字
double calculateTax(double income) {
    if (income > 50000) return income * 0.3;
    if (income > 20000) return income * 0.2;
    return income * 0.1;
}

// AFTER - 命名常量
constexpr double HIGH_TAX_THRESHOLD = 50000.0;
constexpr double MID_TAX_THRESHOLD = 20000.0;
constexpr double HIGH_TAX_RATE = 0.30;
constexpr double MID_TAX_RATE = 0.20;
constexpr double LOW_TAX_RATE = 0.10;

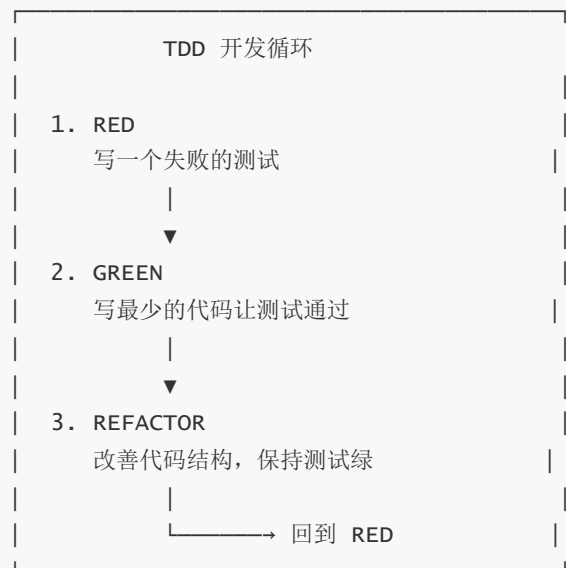
double calculateTax(double income) {
    if (income > HIGH_TAX_THRESHOLD) return income * HIGH_TAX_RATE;
    if (income > MID_TAX_THRESHOLD) return income * MID_TAX_RATE;
    return income * LOW_TAX_RATE;
}
```

Inline Method (内联函数)

```
// BEFORE - 过度提取，只一行代码的函数没有存在必要
int getRating() { return (moreThanFiveLateDeliveries()) ? 2 : 1; }
bool moreThanFiveLateDeliveries() { return lateDeliveries_ > 5; }

// AFTER - 内联回来
int getRating() { return (lateDeliveries_ > 5) ? 2 : 1; }
```

6.3 红-绿-重构循环 (Red-Green-Refactor)



核心原则：

1. 不写测试就不要重构
2. 每次只做一种重构
3. 经常运行测试
4. 小步前进

7. C++ Specific Clean Code

7.1 RAII (Resource Acquisition Is Initialization)

```
// BAD - 手动管理资源
void processFile(const std::string& path) {
    FILE* f = fopen(path.c_str(), "r");
    if (!f) throw std::runtime_error("Cannot open");
    // ... 使用 f
    fclose(f);           // 异常时会泄漏!
}

// GOOD - RAII: 资源绑定到对象生命周期
void processFile(const std::string& path) {
    std::ifstream file(path);
    if (!file) throw std::runtime_error("Cannot open");
    // ... 使用 file
    // 自动关闭, 即使异常也安全
}
```

7.2 Rule of 5 / Rule of 0

```
// Rule of 5: 自定义了任一特殊函数, 就要定义全部 5 个
class ResourceOwner {
public:
    ResourceOwner();           // 构造
    ~ResourceOwner();          // 析构
    ResourceOwner(const ResourceOwner&); // 拷贝构造
    ResourceOwner& operator=(const ResourceOwner&); // 拷贝赋值
    ResourceOwner(ResourceOwner&&);     // 移动构造
    ResourceOwner& operator=(ResourceOwner&&); // 移动赋值
};

// Rule of 0: 最好一个都不要自定义, 让编译器生成
class CleanResourceOwner {
    std::vector<int> data_; // 标准库成员管理自己的资源
    std::unique_ptr<Widget> widget_; // unique_ptr 管理独占资源
    // 编译器生成的拷贝/移动/析构都是正确的
};
```

7.3 const 正确性

```
// BAD - 不写 const
class Account {
public:
    double getBalance() { return balance_; } // 应该 const
};

// GOOD - const 到处正确
class Account {
public:
    double getBalance() const { return balance_; }
    void deposit(double amount) { balance_ += amount; }

    // 参数 const
    void transferTo(Account& to, double amount) const;
    //          ^非const (要修改)      ^const (不修改 this)
    void printStatement(const Account& acc) const;
    //          ^const (不修改参数)    ^const (不修改 this)
private:
    double balance_ = 0;
};
```

7.4 使用自由函数而非成员函数

```
// BAD - 不需要访问 private 的成员函数
class StringUtils {
public:
    static std::string trim(const std::string& s) { /* ... */ }
};

// GOOD - 自由函数减少耦合
namespace string_utils {
    std::string trim(const std::string& s) { /* ... */ }
}
// Scott Meyers: "non-member non-friend functions increase encapsulation"
// 参考 C++ Coding Standards by Sutter & Alexandrescu 第44条
```

7.5 范围 for 循环

```
// BAD
for (size_t i = 0; i < vec.size(); ++i) {
    process(vec[i]);
}

// GOOD
for (const auto& item : vec) {
    process(item);
}
```

```
// 需要修改时
for (auto& item : vec) {
    item.update();
}
```

7.6 auto 的合理使用

```
// GOOD use of auto - 类型明显或冗长
auto it = map.find(key); // 迭代器类型冗长
auto result = std::make_unique<VeryLongType>(); // 右边已有类型
auto lambda = [](int x) { return x * 2; }; // lambda 无显式类型

// BAD use of auto - 类型不明显
auto value = getSomething(); // 什么是 Something?
auto flag = computeFlag(); // 什么类型?

// GOOD instead
double value = getSomething();
bool flag = computeFlag();
```

7.7 nullptr 代替 NULL/0

```
// BAD
void f(int);
void f(char*);
f(NULL); // 调用的是 f(int)! NULL 通常是 #define NULL 0
f(0); // 也是 f(int)

// GOOD
void f(int);
void f(std::nullptr_t) = delete; // 可选: 禁止 nullptr 转 int
f(nullptr); // 明确调用 f(char*) 或 f(nullptr_t)
```

7.8 enum class 代替 enum

```
// BAD - 裸 enum: 全局污染, 隐式转 int
enum Color { RED, GREEN, BLUE };
int x = RED; // 隐式转换, 不类型安全
auto c = Color::RED; // OK, 但 ...
if (x == RED) {} // 不同类型比较, 不报错

// GOOD - enum class: 作用域限定, 强类型
enum class Color { Red, Green, Blue };
int x = static_cast<int>(Color::Red); // 必须显式转换
auto c = Color::Red;
// if (x == Color::Red) {} // 编译错误! 不同类型
```

7.9 综合 C++ 整洁代码示例

```
// =====
// BAD - 包含多种不整洁问题
// =====
class c {
    int d; double e;
public:
    int f(int x, int y, bool z) {
        // NULL 而不是 nullptr
        if (NULL == getPtr()) return -1;
        // 裸 new
        int* buf = new int[1024];
        // 魔法数字
        for (int i = 0; i < 1024; i++) buf[i] = x + y;
        // 布尔参数控制行为
        int result = z ? x + y : x - y;
        // 忘记 delete
        return result;
    }
};

// =====
// GOOD - 整洁的现代 C++
// =====
class Calculator {
public:
    enum class Operation { Add, Subtract };
    explicit Calculator(int initialValue) : value_(initialValue) {}

    [[nodiscard]] int apply(Operation op, int operand) const {
        switch (op) {
            case Operation::Add: return value_ + operand;
            case Operation::Subtract: return value_ - operand;
        }
        return value_;
    }

    [[nodiscard]] std::vector<int> computeBuffer(int other) const {
        constexpr size_t kBufferSize = 1024;
        std::vector<int> buf(kBufferSize);
        for (auto& item : buf) {
            item = value_ + other;
        }
        return buf;
    }

private:
    int value_;
};
```

改进点总结:

- 有意义的类名和成员名
 - `enum class` 代替 `bool` + 隐式 `enum`
 - `constexpr` 代替魔法数字
 - `std::vector` 代替裸 `new[]` (RAII)
 - `const` 成员函数
 - `[[nodiscard]]` 防止忽略返回值
 - 范围 `for` 循环
 - 构造函数 `explicit`
-

8. 面试核心

面试重点

1. **SOLID 原则**: 被问到概率 >90%, 必须能说出每个字母的含义并给出 BAD/GOOD 代码示例
2. **命名规范**: 变量/函数/类命名规则, 能举出 BAD/GOOD 对比
3. **函数设计**: 短小、参数少、无副作用、CQS
4. **RAII 和 Rule of 0/5**: C++ 特有的整洁代码基石
5. **代码异味识别**: 至少能说 10 种并给出改善方案
6. **重构手法**: Extract Method、Replace Conditional with Polymorphism 等核心手法

高频考点

- **SOLID 具体应用**: 给你一段代码, 指出违反哪个原则, 如何修改
- **const 正确性**: 哪些地方该加 `const` (参数、返回值、成员函数、局部变量)
- **RAII vs GC**: C++ 为什么不用 GC, RAII 有什么优势
- **组合 vs 继承**: 什么时候用哪个, 和 SOLID 的关系
- **代码 Review**: 给你一段有问题的代码, 请指出所有问题
- **TDD 和重构的关系**: 红-绿-重构循环

常见陷阱

1. **神圣不可侵犯代码**: "这段代码一直这样写的" — 不能成为不重构的借口
2. **过度设计**: 只有一个子类就搞抽象工厂
3. **重构而不测试**: 没有安全网就重构 ≈ 赌博
4. **完美主义**: 一次想把所有代码变完美 → 永远完不成
5. **const 只写在返回值**: 参数也应该 `const` reference, 成员函数也应该 `const`
6. **不删死代码**: 用注释保留而不是 Git
7. **贫血模型**: 把所有逻辑放 Service 里, Model 只有 getter/setter

建议背诵内容

1. **SOLID 五个字母**: "SRP 单一职责, OCP 开闭原则, LSP 里氏替换, ISP 接口隔离, DIP 依赖倒置"
2. **名称好坏三原则**: "有意义、可搜索、不说谎"
3. **函数好坏三原则**: "短小、做一件事、无副作用"
4. **C++ 整洁代码金句**:
 - o "RAII 是 C++ 最强大的惯用法: 资源绑定到对象, 对象绑定到作用域"
 - o "Rule of 0: 不要写析构函数/拷贝/移动, 让编译器生成"
 - o "const 是合约的一部分: 承诺不修改就写 const"
 - o "enum class 是 C++11 给的类型安全礼物"
5. **重构原则**: "不改变外部行为, 小步前进, 随时测试"
6. **代码异味一句话**: "代码异味不是 Bug, 是技术债的信号。看见就记下来, 定期偿还。"

面试建议: Clean Code 问题的核心是**代码品味**。面试官更可能给你一段烂代码让你挑毛病, 而不是让你背诵原则。训练时多看烂代码, 训练出"看一眼就觉得不对劲"的直觉。